

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Eval Review</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@500;600&family=Lora:wght@4
00;500&display=swap" rel="stylesheet">
  <script src="https://cdn.sheetjs.com/xlsx-0.20.3/package/dist/xlsx.full.min.js"
integrity="sha384-EnyY0/GSHQGSxSgMwaIPzSESbqoOLSexfnSMN2AP+39Ckmn92stwABZynq1JyzdT"
crossorigin="anonymous"></script>
  <style>
    :root {
      --bg: #faf9f5;
      --surface: #ffffff;
      --border: #e8e6dc;
      --text: #141413;
      --text-muted: #b0aea5;
      --accent: #d97757;
      --accent-hover: #c4613f;
      --green: #788c5d;
      --green-bg: #eef2e8;
      --red: #c44;
      --red-bg: #fceaea;
      --header-bg: #141413;
      --header-text: #faf9f5;
      --radius: 6px;
    }

    * { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      font-family: 'Lora', Georgia, serif;
      background: var(--bg);
      color: var(--text);
      height: 100vh;
      display: flex;
      flex-direction: column;
    }

    /* ---- Header ---- */
    .header {
      background: var(--header-bg);
      color: var(--header-text);
      padding: 1rem 2rem;
      display: flex;
      justify-content: space-between;
      align-items: center;
      flex-shrink: 0;
    }

    .header h1 {
      font-family: 'Poppins', sans-serif;

```

```
    font-size: 1.25rem;
    font-weight: 600;
  }
  .header .instructions {
    font-size: 0.8rem;
    opacity: 0.7;
    margin-top: 0.25rem;
  }
  .header .progress {
    font-size: 0.875rem;
    opacity: 0.8;
    text-align: right;
  }
}

/* ---- Main content ---- */
.main {
  flex: 1;
  overflow-y: auto;
  padding: 1.5rem 2rem;
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}

/* ---- Sections ---- */
.section {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  flex-shrink: 0;
}
.section-header {
  font-family: 'Poppins', sans-serif;
  padding: 0.75rem 1rem;
  font-size: 0.75rem;
  font-weight: 500;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--text-muted);
  border-bottom: 1px solid var(--border);
  background: var(--bg);
}
.section-body {
  padding: 1rem;
}

/* ---- Config badge ---- */
.config-badge {
  display: inline-block;
  padding: 0.2rem 0.625rem;
  border-radius: 9999px;
  font-family: 'Poppins', sans-serif;
  font-size: 0.6875rem;
  font-weight: 600;
  text-transform: uppercase;
}
```

```

    letter-spacing: 0.03em;
    margin-left: 0.75rem;
    vertical-align: middle;
}
.config-badge.config-primary {
    background: rgba(33, 150, 243, 0.12);
    color: #1976d2;
}
.config-badge.config-baseline {
    background: rgba(255, 193, 7, 0.15);
    color: #f57f17;
}

/* ---- Prompt ---- */
.prompt-text {
    white-space: pre-wrap;
    font-size: 0.9375rem;
    line-height: 1.6;
}

/* ---- Outputs ---- */
.output-file {
    border: 1px solid var(--border);
    border-radius: var(--radius);
    overflow: hidden;
}
.output-file + .output-file {
    margin-top: 1rem;
}
.output-file-header {
    padding: 0.5rem 0.75rem;
    font-size: 0.8rem;
    font-weight: 600;
    color: var(--text-muted);
    background: var(--bg);
    border-bottom: 1px solid var(--border);
    font-family: 'SF Mono', SFMono-Regular, Consolas, 'Liberation Mono', Menlo,
monospace;
    display: flex;
    justify-content: space-between;
    align-items: center;
}
.output-file-header .dl-btn {
    font-size: 0.7rem;
    color: var(--accent);
    text-decoration: none;
    cursor: pointer;
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
    font-weight: 500;
    opacity: 0.8;
}
.output-file-header .dl-btn:hover {
    opacity: 1;
    text-decoration: underline;
}

```

```
.output-file-content {
  padding: 0.75rem;
  overflow-x: auto;
}
.output-file-content pre {
  font-size: 0.8125rem;
  line-height: 1.5;
  white-space: pre-wrap;
  word-break: break-word;
  font-family: 'SF Mono', SFMono-Regular, Consolas, 'Liberation Mono', Menlo,
monospace;
}
.output-file-content img {
  max-width: 100%;
  height: auto;
  border-radius: 4px;
}
.output-file-content iframe {
  width: 100%;
  height: 600px;
  border: none;
}
.output-file-content table {
  border-collapse: collapse;
  font-size: 0.8125rem;
  width: 100%;
}
.output-file-content table td,
.output-file-content table th {
  border: 1px solid var(--border);
  padding: 0.375rem 0.5rem;
  text-align: left;
}
.output-file-content table th {
  background: var(--bg);
  font-weight: 600;
}
.output-file-content .download-link {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  padding: 0.5rem 1rem;
  background: var(--bg);
  border: 1px solid var(--border);
  border-radius: 4px;
  color: var(--accent);
  text-decoration: none;
  font-size: 0.875rem;
  cursor: pointer;
}
.output-file-content .download-link:hover {
  background: var(--border);
}
.empty-state {
  color: var(--text-muted);
```

```
    font-style: italic;
    padding: 2rem;
    text-align: center;
}

/* ---- Feedback ---- */
.prev-feedback {
    background: var(--bg);
    border: 1px solid var(--border);
    border-radius: 4px;
    padding: 0.625rem 0.75rem;
    margin-top: 0.75rem;
    font-size: 0.8125rem;
    color: var(--text-muted);
    line-height: 1.5;
}
.prev-feedback-label {
    font-size: 0.7rem;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.04em;
    margin-bottom: 0.25rem;
    color: var(--text-muted);
}
.feedback-textarea {
    width: 100%;
    min-height: 100px;
    padding: 0.75rem;
    border: 1px solid var(--border);
    border-radius: 4px;
    font-family: inherit;
    font-size: 0.9375rem;
    line-height: 1.5;
    resize: vertical;
    color: var(--text);
}
.feedback-textarea:focus {
    outline: none;
    border-color: var(--accent);
    box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.1);
}
.feedback-status {
    font-size: 0.75rem;
    color: var(--text-muted);
    margin-top: 0.5rem;
    min-height: 1.1em;
}

/* ---- Grades (collapsible) ---- */
.grades-toggle {
    display: flex;
    align-items: center;
    cursor: pointer;
    user-select: none;
}
```

```
.grades-toggle:hover {
  color: var(--accent);
}
.grades-toggle .arrow {
  margin-right: 0.5rem;
  transition: transform 0.15s;
  font-size: 0.75rem;
}
.grades-toggle .arrow.open {
  transform: rotate(90deg);
}
.grades-content {
  display: none;
  margin-top: 0.75rem;
}
.grades-content.open {
  display: block;
}
.grades-summary {
  font-size: 0.875rem;
  margin-bottom: 0.75rem;
  display: flex;
  align-items: center;
  gap: 0.5rem;
}
.grade-badge {
  display: inline-block;
  padding: 0.125rem 0.5rem;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 600;
}
.grade-pass { background: var(--green-bg); color: var(--green); }
.grade-fail { background: var(--red-bg); color: var(--red); }
.assertion-list {
  list-style: none;
}
.assertion-item {
  padding: 0.625rem 0;
  border-bottom: 1px solid var(--border);
  font-size: 0.8125rem;
}
.assertion-item:last-child { border-bottom: none; }
.assertion-status {
  font-weight: 600;
  margin-right: 0.5rem;
}
.assertion-status.pass { color: var(--green); }
.assertion-status.fail { color: var(--red); }
.assertion-evidence {
  color: var(--text-muted);
  font-size: 0.75rem;
  margin-top: 0.25rem;
  padding-left: 1.5rem;
}
```

```

/* ---- View tabs ---- */
.view-tabs {
  display: flex;
  gap: 0;
  padding: 0 2rem;
  background: var(--bg);
  border-bottom: 1px solid var(--border);
  flex-shrink: 0;
}
.view-tab {
  font-family: 'Poppins', sans-serif;
  padding: 0.625rem 1.25rem;
  font-size: 0.8125rem;
  font-weight: 500;
  cursor: pointer;
  border: none;
  background: none;
  color: var(--text-muted);
  border-bottom: 2px solid transparent;
  transition: all 0.15s;
}
.view-tab:hover { color: var(--text); }
.view-tab.active {
  color: var(--accent);
  border-bottom-color: var(--accent);
}
.view-panel { display: none; }
.view-panel.active { display: flex; flex-direction: column; flex: 1; overflow:
hidden; }

/* ---- Benchmark view ---- */
.benchmark-view {
  padding: 1.5rem 2rem;
  overflow-y: auto;
  flex: 1;
}
.benchmark-table {
  border-collapse: collapse;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  font-size: 0.8125rem;
  width: 100%;
  margin-bottom: 1.5rem;
}
.benchmark-table th, .benchmark-table td {
  padding: 0.625rem 0.75rem;
  text-align: left;
  border: 1px solid var(--border);
}
.benchmark-table th {
  font-family: 'Poppins', sans-serif;
  background: var(--header-bg);
  color: var(--header-text);
}

```

```

    font-weight: 500;
    font-size: 0.75rem;
    text-transform: uppercase;
    letter-spacing: 0.04em;
}
.benchmark-table tr:hover { background: var(--bg); }
.benchmark-table tr.benchmark-row-with { background: rgba(33, 150, 243, 0.06); }
.benchmark-table tr.benchmark-row-without { background: rgba(255, 193, 7, 0.06); }
.benchmark-table tr.benchmark-row-with:hover { background: rgba(33, 150, 243,
0.12); }
.benchmark-table tr.benchmark-row-without:hover { background: rgba(255, 193, 7,
0.12); }
.benchmark-table tr.benchmark-row-avg { font-weight: 600; border-top: 2px solid
var(--border); }
.benchmark-table tr.benchmark-row-avg.benchmark-row-with { background: rgba(33,
150, 243, 0.12); }
.benchmark-table tr.benchmark-row-avg.benchmark-row-without { background: rgba(255,
193, 7, 0.12); }
.benchmark-delta-positive { color: var(--green); font-weight: 600; }
.benchmark-delta-negative { color: var(--red); font-weight: 600; }
.benchmark-notes {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: var(--radius);
    padding: 1rem;
}
.benchmark-notes h3 {
    font-family: 'Poppins', sans-serif;
    font-size: 0.875rem;
    margin-bottom: 0.75rem;
}
.benchmark-notes ul {
    list-style: disc;
    padding-left: 1.25rem;
}
.benchmark-notes li {
    font-size: 0.8125rem;
    line-height: 1.6;
    margin-bottom: 0.375rem;
}
.benchmark-empty {
    color: var(--text-muted);
    font-style: italic;
    text-align: center;
    padding: 3rem;
}

/* ---- Navigation ---- */
.nav {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 1rem 2rem;
    border-top: 1px solid var(--border);
    background: var(--surface);

```



```
    flex-shrink: 0;
  }
  .nav-btn {
    font-family: 'Poppins', sans-serif;
    padding: 0.5rem 1.25rem;
    border: 1px solid var(--border);
    border-radius: var(--radius);
    background: var(--surface);
    cursor: pointer;
    font-size: 0.875rem;
    font-weight: 500;
    color: var(--text);
    transition: all 0.15s;
  }
  .nav-btn:hover:not(:disabled) {
    background: var(--bg);
    border-color: var(--text-muted);
  }
  .nav-btn:disabled {
    opacity: 0.4;
    cursor: not-allowed;
  }
  .done-btn {
    font-family: 'Poppins', sans-serif;
    padding: 0.5rem 1.5rem;
    border: 1px solid var(--border);
    border-radius: var(--radius);
    background: var(--surface);
    color: var(--text);
    cursor: pointer;
    font-size: 0.875rem;
    font-weight: 500;
    transition: all 0.15s;
  }
  .done-btn:hover {
    background: var(--bg);
    border-color: var(--text-muted);
  }
  .done-btn.ready {
    border: none;
    background: var(--accent);
    color: white;
    font-weight: 600;
  }
  .done-btn.ready:hover {
    background: var(--accent-hover);
  }
  /* ---- Done overlay ---- */
  .done-overlay {
    display: none;
    position: fixed;
    inset: 0;
    background: rgba(0, 0, 0, 0.5);
    z-index: 100;
    justify-content: center;
```

```

    align-items: center;
}
.done-overlay.visible {
    display: flex;
}
.done-card {
    background: var(--surface);
    border-radius: 12px;
    padding: 2rem 3rem;
    text-align: center;
    box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);
    max-width: 500px;
}
.done-card h2 {
    font-size: 1.5rem;
    margin-bottom: 0.5rem;
}
.done-card p {
    color: var(--text-muted);
    margin-bottom: 1.5rem;
    line-height: 1.5;
}
.done-card .btn-row {
    display: flex;
    gap: 0.5rem;
    justify-content: center;
}
.done-card button {
    padding: 0.5rem 1.25rem;
    border: 1px solid var(--border);
    border-radius: var(--radius);
    background: var(--surface);
    cursor: pointer;
    font-size: 0.875rem;
}
.done-card button:hover {
    background: var(--bg);
}
/* ---- Toast ---- */
.toast {
    position: fixed;
    bottom: 5rem;
    left: 50%;
    transform: translateX(-50%);
    background: var(--header-bg);
    color: var(--header-text);
    padding: 0.625rem 1.25rem;
    border-radius: var(--radius);
    font-size: 0.875rem;
    opacity: 0;
    transition: opacity 0.3s;
    pointer-events: none;
    z-index: 200;
}
.toast.visible {

```

```

opacity: 1;
}
</style>
</head>
<body>
  <div id="app" style="height:100vh; display:flex; flex-direction:column;">
    <div class="header">
      <div>
        <h1>Eval Review: <span id="skill-name"></span></h1>
        <div class="instructions">Review each output and leave feedback below. Navigate
with arrow keys or buttons. When done, copy feedback and paste into Claude Code.</div>
      </div>
      <div class="progress" id="progress"></div>
    </div>

    <!-- View tabs (only shown when benchmark data exists) -->
    <div class="view-tabs" id="view-tabs" style="display:none;">
      <button class="view-tab active" onclick="switchView('outputs')">Outputs</button>
      <button class="view-tab" onclick="switchView('benchmark')">Benchmark</button>
    </div>

    <!-- Outputs panel (qualitative review) -->
    <div class="view-panel active" id="panel-outputs">
      <div class="main">
        <!-- Prompt -->
        <div class="section">
          <div class="section-header">Prompt <span class="config-badge" id="config-badge"
style="display:none;"></span></div>
          <div class="section-body">
            <div class="prompt-text" id="prompt-text"></div>
          </div>
        </div>

        <!-- Outputs -->
        <div class="section">
          <div class="section-header">Output</div>
          <div class="section-body" id="outputs-body">
            <div class="empty-state">No output files found</div>
          </div>
        </div>

        <!-- Previous Output (collapsible) -->
        <div class="section" id="prev-outputs-section" style="display:none;">
          <div class="section-header">
            <div class="grades-toggle" onclick="togglePrevOutputs()">
              <span class="arrow" id="prev-outputs-arrow">&#9654;</span>
              Previous Output
            </div>
          </div>
          <div class="grades-content" id="prev-outputs-content"></div>
        </div>

        <!-- Grades (collapsible) -->
        <div class="section" id="grades-section" style="display:none;">
          <div class="section-header">

```

```

        <div class="grades-toggle" onclick="toggleGrades()">
            <span class="arrow" id="grades-arrow">&#9654;</span>
            Formal Grades
        </div>
    </div>
    <div class="grades-content" id="grades-content"></div>
</div>

<!-- Feedback -->
<div class="section">
    <div class="section-header">Your Feedback</div>
    <div class="section-body">
        <textarea
            class="feedback-textarea"
            id="feedback"
            placeholder="What do you think of this output? Any issues, suggestions, or
things that look great?"
        ></textarea>
        <div class="feedback-status" id="feedback-status"></div>
        <div class="prev-feedback" id="prev-feedback" style="display:none;">
            <div class="prev-feedback-label">Previous feedback</div>
            <div id="prev-feedback-text"></div>
        </div>
    </div>
</div>
</div>

<div class="nav" id="outputs-nav">
    <button class="nav-btn" id="prev-btn" onclick="navigate(-1)">&#8592;
Previous</button>
    <button class="done-btn" id="done-btn" onclick="showDoneDialog()">Submit All
Reviews</button>
    <button class="nav-btn" id="next-btn" onclick="navigate(1)">Next &#8594;</button>
</div>
</div><!-- end panel-outputs -->

<!-- Benchmark panel (quantitative stats) -->
<div class="view-panel" id="panel-benchmark">
    <div class="benchmark-view" id="benchmark-content">
        <div class="benchmark-empty">No benchmark data available. Run a benchmark to
see quantitative results here.</div>
    </div>
</div>
</div>

<!-- Done overlay -->
<div class="done-overlay" id="done-overlay">
    <div class="done-card">
        <h2>Review Complete</h2>
        <p>Your feedback has been saved. Go back to your Claude Code session and tell
Claude you're done reviewing.</p>
        <div class="btn-row">
            <button onclick="closeDoneDialog()">OK</button>
        </div>
    </div>
</div>

```

```

</div>

<!-- Toast -->
<div class="toast" id="toast"></div>

<script>
  // ---- Embedded data (injected by generate_review.py) ----
  /*__EMBEDDED_DATA__*/

  // ---- State ----
  let feedbackMap = {}; // run_id -> feedback text
  let currentIndex = 0;
  let visitedRuns = new Set();

  // ---- Init ----
  async function init() {
    // Load saved feedback from server – but only if this isn't a fresh
    // iteration (indicated by previous_feedback being present). When
    // previous feedback exists, the feedback.json on disk is stale from
    // the prior iteration and should not pre-fill the textareas.
    const hasPrevious = Object.keys(EMBEDDED_DATA.previous_feedback || {}).length > 0
      || Object.keys(EMBEDDED_DATA.previous_outputs || {}).length > 0;
    if (!hasPrevious) {
      try {
        const resp = await fetch("/api/feedback");
        const data = await resp.json();
        if (data.reviews) {
          for (const r of data.reviews) feedbackMap[r.run_id] = r.feedback;
        }
      } catch { /* first run, no feedback yet */ }
    }

    document.getElementById("skill-name").textContent = EMBEDDED_DATA.skill_name;
    showRun(0);

    // Wire up feedback auto-save
    const textarea = document.getElementById("feedback");
    let saveTimeout = null;
    textarea.addEventListener("input", () => {
      clearTimeout(saveTimeout);
      document.getElementById("feedback-status").textContent = "";
      saveTimeout = setTimeout(() => saveCurrentFeedback(), 800);
    });
  }

  // ---- Navigation ----
  function navigate(delta) {
    const newIndex = currentIndex + delta;
    if (newIndex >= 0 && newIndex < EMBEDDED_DATA.runs.length) {
      saveCurrentFeedback();
      showRun(newIndex);
    }
  }

  function updateNavButtons() {

```

```

document.getElementById("prev-btn").disabled = currentIndex === 0;
document.getElementById("next-btn").disabled =
  currentIndex === EMBEDDED_DATA.runs.length - 1;
}

// ---- Show a run ----
function showRun(index) {
  currentIndex = index;
  const run = EMBEDDED_DATA.runs[index];

  // Progress
  document.getElementById("progress").textContent =
    `${index + 1} of ${EMBEDDED_DATA.runs.length}`;

  // Prompt
  document.getElementById("prompt-text").textContent = run.prompt;

  // Config badge
  const badge = document.getElementById("config-badge");
  const configMatch =
run.id.match(/(with_skill|without_skill|new_skill|old_skill)/);
  if (configMatch) {
    const config = configMatch[1];
    const isBaseline = config === "without_skill" || config === "old_skill";
    badge.textContent = config.replace(/_/g, " ");
    badge.className = "config-badge " + (isBaseline ? "config-baseline" :
"config-primary");
    badge.style.display = "inline-block";
  } else {
    badge.style.display = "none";
  }

  // Outputs
  renderOutputs(run);

  // Previous outputs
  renderPrevOutputs(run);

  // Grades
  renderGrades(run);

  // Previous feedback
  const prevFb = (EMBEDDED_DATA.previous_feedback || {})[run.id];
  const prevEl = document.getElementById("prev-feedback");
  if (prevFb) {
    document.getElementById("prev-feedback-text").textContent = prevFb;
    prevEl.style.display = "block";
  } else {
    prevEl.style.display = "none";
  }

  // Feedback
  document.getElementById("feedback").value = feedbackMap[run.id] || "";
  document.getElementById("feedback-status").textContent = "";

```

```

updateNavButtons();

// Track visited runs and promote done button when all visited
visitedRuns.add(index);
const doneBtn = document.getElementById("done-btn");
if (visitedRuns.size >= EMBEDDED_DATA.runs.length) {
    doneBtn.classList.add("ready");
}

// Scroll main content to top
document.querySelector(".main").scrollTop = 0;
}

// ---- Render outputs ----
function renderOutputs(run) {
    const container = document.getElementById("outputs-body");
    container.innerHTML = "";

    const outputs = run.outputs || [];
    if (outputs.length === 0) {
        container.innerHTML = '<div class="empty-state">No output files</div>';
        return;
    }

    for (const file of outputs) {
        const fileDiv = document.createElement("div");
        fileDiv.className = "output-file";

        // Always show file header with download link
        const header = document.createElement("div");
        header.className = "output-file-header";
        const nameSpan = document.createElement("span");
        nameSpan.textContent = file.name;
        header.appendChild(nameSpan);
        const dlBtn = document.createElement("a");
        dlBtn.className = "dl-btn";
        dlBtn.textContent = "Download";
        dlBtn.download = file.name;
        dlBtn.href = getDownloadUri(file);
        header.appendChild(dlBtn);
        fileDiv.appendChild(header);

        const content = document.createElement("div");
        content.className = "output-file-content";

        if (file.type === "text") {
            const pre = document.createElement("pre");
            pre.textContent = file.content;
            content.appendChild(pre);
        } else if (file.type === "image") {
            const img = document.createElement("img");
            img.src = file.data_uri;
            img.alt = file.name;
            content.appendChild(img);
        } else if (file.type === "pdf") {

```

```

        const iframe = document.createElement("iframe");
        iframe.src = file.data_uri;
        content.appendChild(iframe);
    } else if (file.type === "xlsx") {
        renderXlsx(content, file.data_b64);
    } else if (file.type === "binary") {
        const a = document.createElement("a");
        a.className = "download-link";
        a.href = file.data_uri;
        a.download = file.name;
        a.textContent = "Download " + file.name;
        content.appendChild(a);
    } else if (file.type === "error") {
        const pre = document.createElement("pre");
        pre.textContent = file.content;
        pre.style.color = "var(--red)";
        content.appendChild(pre);
    }

    fileDiv.appendChild(content);
    container.appendChild(fileDiv);
}
}

// ---- XLSX rendering via SheetJS ----
function renderXlsx(container, b64Data) {
    try {
        const raw = Uint8Array.from(atob(b64Data), c => c.charCodeAt(0));
        const wb = XLSX.read(raw, { type: "array" });

        for (let i = 0; i < wb.SheetNames.length; i++) {
            const sheetName = wb.SheetNames[i];
            const ws = wb.Sheets[sheetName];

            if (wb.SheetNames.length > 1) {
                const sheetLabel = document.createElement("div");
                sheetLabel.style.cssText =
                    "font-weight:600; font-size:0.8rem; color:#b0aea5; margin-top:0.5rem; margin-bottom:0.25rem;";
                sheetLabel.textContent = "Sheet: " + sheetName;
                container.appendChild(sheetLabel);
            }

            const htmlStr = XLSX.utils.sheet_to_html(ws, { editable: false });
            const wrapper = document.createElement("div");
            wrapper.innerHTML = htmlStr;
            container.appendChild(wrapper);
        }
    } catch (err) {
        container.textContent = "Error rendering spreadsheet: " + err.message;
    }
}

// ---- Grades ----
function renderGrades(run) {

```



```

const section = document.getElementById("grades-section");
const content = document.getElementById("grades-content");

if (!run.grading) {
  section.style.display = "none";
  return;
}

const grading = run.grading;
section.style.display = "block";
// Reset to collapsed
content.classList.remove("open");
document.getElementById("grades-arrow").classList.remove("open");

const summary = grading.summary || {};
const expectations = grading.expectations || [];

let html = '<div style="padding: 1rem;">';

// Summary line
const passRate = summary.pass_rate != null
  ? Math.round(summary.pass_rate * 100) + "%"
  : "?";
const badgeClass = summary.pass_rate >= 0.8 ? "grade-pass" : summary.pass_rate >=
0.5 ? "" : "grade-fail";
html += '<div class="grades-summary">';
html += '<span class="grade-badge ' + badgeClass + '">' + passRate + '</span>';
html += '<span>' + (summary.passed || 0) + ' passed, ' + (summary.failed || 0) +
' failed of ' + (summary.total || 0) + '</span>';
html += '</div>';

// Assertions list
html += '<ul class="assertion-list">';
for (const exp of expectations) {
  const statusClass = exp.passed ? "pass" : "fail";
  const statusIcon = exp.passed ? "\u2713" : "\u2717";
  html += '<li class="assertion-item">';
  html += '<span class="assertion-status ' + statusClass + '">' + statusIcon +
'</span>';
  html += '<span>' + escapeHtml(exp.text) + '</span>';
  if (exp.evidence) {
    html += '<div class="assertion-evidence">' + escapeHtml(exp.evidence) +
'</div>';
  }
  html += '</li>';
}
html += '</ul>';

html += '</div>';
content.innerHTML = html;
}

function toggleGrades() {
  const content = document.getElementById("grades-content");
  const arrow = document.getElementById("grades-arrow");

```

```

    content.classList.toggle("open");
    arrow.classList.toggle("open");
}

// ---- Previous outputs (collapsible) ----
function renderPrevOutputs(run) {
    const section = document.getElementById("prev-outputs-section");
    const content = document.getElementById("prev-outputs-content");
    const prevOutputs = (EMBEDDED_DATA.previous_outputs || {})[run.id];

    if (!prevOutputs || prevOutputs.length === 0) {
        section.style.display = "none";
        return;
    }

    section.style.display = "block";
    // Reset to collapsed
    content.classList.remove("open");
    document.getElementById("prev-outputs-arrow").classList.remove("open");

    // Render the files into the content area
    content.innerHTML = "";
    const wrapper = document.createElement("div");
    wrapper.style.padding = "1rem";

    for (const file of prevOutputs) {
        const fileDiv = document.createElement("div");
        fileDiv.className = "output-file";

        const header = document.createElement("div");
        header.className = "output-file-header";
        const nameSpan = document.createElement("span");
        nameSpan.textContent = file.name;
        header.appendChild(nameSpan);
        const dlBtn = document.createElement("a");
        dlBtn.className = "dl-btn";
        dlBtn.textContent = "Download";
        dlBtn.download = file.name;
        dlBtn.href = getDownloadUri(file);
        header.appendChild(dlBtn);
        fileDiv.appendChild(header);

        const fc = document.createElement("div");
        fc.className = "output-file-content";

        if (file.type === "text") {
            const pre = document.createElement("pre");
            pre.textContent = file.content;
            fc.appendChild(pre);
        } else if (file.type === "image") {
            const img = document.createElement("img");
            img.src = file.data_uri;
            img.alt = file.name;
            fc.appendChild(img);
        } else if (file.type === "pdf") {

```

```

        const iframe = document.createElement("iframe");
        iframe.src = file.data_uri;
        fc.appendChild(iframe);
    } else if (file.type === "xlsx") {
        renderXlsx(fc, file.data_b64);
    } else if (file.type === "binary") {
        const a = document.createElement("a");
        a.className = "download-link";
        a.href = file.data_uri;
        a.download = file.name;
        a.textContent = "Download " + file.name;
        fc.appendChild(a);
    }

    fileDiv.appendChild(fc);
    wrapper.appendChild(fileDiv);
}

content.appendChild(wrapper);
}

function togglePrevOutputs() {
    const content = document.getElementById("prev-outputs-content");
    const arrow = document.getElementById("prev-outputs-arrow");
    content.classList.toggle("open");
    arrow.classList.toggle("open");
}

// ---- Feedback (saved to server -> feedback.json) ----
function saveCurrentFeedback() {
    const run = EMBEDDED_DATA.runs[currentIndex];
    const text = document.getElementById("feedback").value;

    if (text.trim() === "") {
        delete feedbackMap[run.id];
    } else {
        feedbackMap[run.id] = text;
    }

    // Build reviews array from map
    const reviews = [];
    for (const [run_id, feedback] of Object.entries(feedbackMap)) {
        if (feedback.trim()) {
            reviews.push({ run_id, feedback, timestamp: new Date().toISOString() });
        }
    }

    fetch("/api/feedback", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ reviews, status: "in_progress" }),
    }).then(() => {
        document.getElementById("feedback-status").textContent = "Saved";
    }).catch(() => {
        // Static mode or server unavailable - no-op on auto-save,

```

```

        // feedback will be downloaded on final submit
        document.getElementById("feedback-status").textContent = "Will download on
submit";
    });
}

// ---- Done ----
function showDoneDialog() {
    // Save current textarea to feedbackMap (but don't POST yet)
    const run = EMBEDDED_DATA.runs[currentIndex];
    const text = document.getElementById("feedback").value;
    if (text.trim() === "") {
        delete feedbackMap[run.id];
    } else {
        feedbackMap[run.id] = text;
    }

    // POST once with status: complete — include ALL runs so the model
    // can distinguish "no feedback" (looks good) from "not reviewed"
    const reviews = [];
    const ts = new Date().toISOString();
    for (const r of EMBEDDED_DATA.runs) {
        reviews.push({ run_id: r.id, feedback: feedbackMap[r.id] || "", timestamp: ts
});
    }
    const payload = JSON.stringify({ reviews, status: "complete" }, null, 2);
    fetch("/api/feedback", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: payload,
    }).then(() => {
        document.getElementById("done-overlay").classList.add("visible");
    }).catch(() => {
        // Server not available (static mode) — download as file
        const blob = new Blob([payload], { type: "application/json" });
        const url = URL.createObjectURL(blob);
        const a = document.createElement("a");
        a.href = url;
        a.download = "feedback.json";
        a.click();
        URL.revokeObjectURL(url);
        document.getElementById("done-overlay").classList.add("visible");
    });
}

function closeDoneDialog() {
    // Reset status back to in_progress
    saveCurrentFeedback();
    document.getElementById("done-overlay").classList.remove("visible");
}

// ---- Toast ----
function showToast(message) {
    const toast = document.getElementById("toast");
    toast.textContent = message;
}

```

```

    toast.classList.add("visible");
    setTimeout(() => toast.classList.remove("visible"), 2000);
}

// ---- Keyboard nav ----
document.addEventListener("keydown", (e) => {
    // Don't capture when typing in textarea
    if (e.target.tagName === "TEXTAREA") return;

    if (e.key === "ArrowLeft" || e.key === "ArrowUp") {
        e.preventDefault();
        navigate(-1);
    } else if (e.key === "ArrowRight" || e.key === "ArrowDown") {
        e.preventDefault();
        navigate(1);
    }
});

// ---- Util ----
function getDownloadUri(file) {
    if (file.data_uri) return file.data_uri;
    if (file.data_b64) return "data:application/octet-stream;base64," +
file.data_b64;
    if (file.type === "text") return "data:text/plain;charset=utf-8," +
encodeURIComponent(file.content);
    return "#";
}

function escapeHtml(text) {
    const div = document.createElement("div");
    div.textContent = text;
    return div.innerHTML;
}

// ---- View switching ----
function switchView(view) {
    document.querySelectorAll(".view-tab").forEach(t =>
t.classList.remove("active"));
    document.querySelectorAll(".view-panel").forEach(p =>
p.classList.remove("active"));

document.querySelector(`[onclick="switchView('${view}')]`).classList.add("active");
    document.getElementById("panel-" + view).classList.add("active");
}

// ---- Benchmark rendering ----
function renderBenchmark() {
    const data = EMBEDDED_DATA.benchmark;
    if (!data) return;

    // Show the tabs
    document.getElementById("view-tabs").style.display = "flex";

    const container = document.getElementById("benchmark-content");
    const summary = data.run_summary || {};

```

```

const metadata = data.metadata || {};
const notes = data.notes || [];

let html = "";

// Header
html += "<h2 style='font-family: Poppins, sans-serif; margin-bottom: 0.5rem;'>Benchmark Results</h2>";
html += "<p style='color: var(--text-muted); font-size: 0.875rem; margin-bottom: 1.25rem;'>";
if (metadata.skill_name) html += "<strong>" + escapeHtml(metadata.skill_name) + "</strong> &mdash; ";
if (metadata.timestamp) html += metadata.timestamp + " &mdash; ";
if (metadata.evals_run) html += "Evals: " + metadata.evals_run.join(", ") + " &mdash; ";
html += (metadata.runs_per_configuration || "?") + " runs per configuration";
html += "</p>";

// Summary table
html += '<table class="benchmark-table">';

function fmtStat(stat, pct) {
  if (!stat) return "-";
  const suffix = pct ? "%" : "";
  const m = pct ? (stat.mean * 100).toFixed(0) : stat.mean.toFixed(1);
  const s = pct ? (stat.stddev * 100).toFixed(0) : stat.stddev.toFixed(1);
  return m + suffix + " ± " + s + suffix;
}

function deltaClass(val) {
  if (!val) return "";
  const n = parseFloat(val);
  if (n > 0) return "benchmark-delta-positive";
  if (n < 0) return "benchmark-delta-negative";
  return "";
}

// Discover config names dynamically (everything except "delta")
const configs = Object.keys(summary).filter(k => k !== "delta");
const configA = configs[0] || "config_a";
const configB = configs[1] || "config_b";
const labelA = configA.replace(/_/g, " ").replace(/\b\w/g, c => c.toUpperCase());
const labelB = configB.replace(/_/g, " ").replace(/\b\w/g, c => c.toUpperCase());
const a = summary[configA] || {};
const b = summary[configB] || {};
const delta = summary.delta || {};

html += "<thead><tr><th>Metric</th><th>" + escapeHtml(labelA) + "</th><th>" + escapeHtml(labelB) + "</th><th>Delta</th></tr></thead>";
html += "<tbody>";

html += "<tr><td><strong>Pass Rate</strong></td>";
html += "<td>" + fmtStat(a.pass_rate, true) + "</td>";
html += "<td>" + fmtStat(b.pass_rate, true) + "</td>";

```

```

    html += '<td class="' + deltaClass(delta.pass_rate) + '>' + (delta.pass_rate ||
    "-") + "</td></tr>";

    // Time (only show row if data exists)
    if (a.time_seconds || b.time_seconds) {
        html += "<tr><td><strong>Time (s)</strong></td>";
        html += "<td>" + fmtStat(a.time_seconds, false) + "</td>";
        html += "<td>" + fmtStat(b.time_seconds, false) + "</td>";
        html += '<td class="' + deltaClass(delta.time_seconds) + '>' +
(delta.time_seconds ? delta.time_seconds + "s" : "-") + "</td></tr>";
    }

    // Tokens (only show row if data exists)
    if (a.tokens || b.tokens) {
        html += "<tr><td><strong>Tokens</strong></td>";
        html += "<td>" + fmtStat(a.tokens, false) + "</td>";
        html += "<td>" + fmtStat(b.tokens, false) + "</td>";
        html += '<td class="' + deltaClass(delta.tokens) + '>' + (delta.tokens || "-")
+ "</td></tr>";
    }

    html += "</tbody></table>";

    // Per-eval breakdown (if runs data available)
    const runs = data.runs || [];
    if (runs.length > 0) {
        const evalIds = [...new Set(runs.map(r => r.eval_id))].sort((a, b) => a - b);

        html += "<h3 style='font-family: Poppins, sans-serif; margin-bottom:
0.75rem;'>Per-Eval Breakdown</h3>";

        const hasTime = runs.some(r => r.result && r.result.time_seconds != null);
        const hasErrors = runs.some(r => r.result && r.result.errors > 0);

        for (const evalId of evalIds) {
            const evalRuns = runs.filter(r => r.eval_id === evalId);
            const evalName = evalRuns[0] && evalRuns[0].eval_name ? evalRuns[0].eval_name
: "Eval " + evalId;

            html += "<h4 style='font-family: Poppins, sans-serif; margin: 1rem 0 0.5rem;
color: var(--text);'>" + escapeHtml(evalName) + "</h4>";
            html += '<table class="benchmark-table">';
            html += "<thead><tr><th>Config</th><th>Run</th><th>Pass Rate</th>";
            if (hasTime) html += "<th>Time (s)</th>";
            if (hasErrors) html += "<th>Crashes During Execution</th>";
            html += "</tr></thead>";
            html += "<tbody>";

            // Group by config and render with average rows
            const configGroups = [...new Set(evalRuns.map(r => r.configuration))];
            for (let ci = 0; ci < configGroups.length; ci++) {
                const config = configGroups[ci];
                const configRuns = evalRuns.filter(r => r.configuration === config);
                if (configRuns.length === 0) continue;

```

```

        const rowClass = ci === 0 ? "benchmark-row-with" : "benchmark-row-without";
        const configLabel = config.replace(/_/g, " ").replace(/\b\w/g, c =>
c.toUpperCase());

        for (const run of configRuns) {
            const r = run.result || {};
            const prClass = r.pass_rate >= 0.8 ? "benchmark-delta-positive" :
r.pass_rate < 0.5 ? "benchmark-delta-negative" : "";
            html += '<tr class="' + rowClass + '>';
            html += "<td>" + configLabel + "</td>";
            html += "<td>" + run.run_number + "</td>";
            html += '<td class="' + prClass + '>' + ((r.pass_rate || 0) *
100).toFixed(0) + "% (" + (r.passed || 0) + "/" + (r.total || 0) + ")</td>";
            if (hasTime) html += "<td>" + (r.time_seconds != null ?
r.time_seconds.toFixed(1) : "-") + "</td>";
            if (hasErrors) html += "<td>" + (r.errors || 0) + "</td>";
            html += "</tr>";
        }

        // Average row
        const rates = configRuns.map(r => (r.result || {}).pass_rate || 0);
        const avgRate = rates.reduce((a, b) => a + b, 0) / rates.length;
        const avgPrClass = avgRate >= 0.8 ? "benchmark-delta-positive" : avgRate <
0.5 ? "benchmark-delta-negative" : "";
        html += '<tr class="benchmark-row-avg ' + rowClass + '>';
        html += "<td>" + configLabel + "</td>";
        html += "<td>Avg</td>";
        html += '<td class="' + avgPrClass + '>' + (avgRate * 100).toFixed(0) +
"%</td>";

        if (hasTime) {
            const times = configRuns.map(r => (r.result || {}).time_seconds).filter(t
=> t != null);
            html += "<td>" + (times.length ? (times.reduce((a, b) => a + b, 0) /
times.length).toFixed(1) : "-") + "</td>";
        }
        if (hasErrors) html += "<td></td>";
        html += "</tr>";
    }
    html += "</tbody></table>";

    // Per-assertion detail for this eval
    const runsWithExpectations = {};
    for (const config of configGroups) {
        runsWithExpectations[config] = evalRuns.filter(r => r.configuration ===
config && r.expectations && r.expectations.length > 0);
    }
    const hasAnyExpectations = Object.values(runsWithExpectations).some(runs =>
runs.length > 0);
    if (hasAnyExpectations) {
        // Collect all unique assertion texts across all configs
        const allAssertions = [];
        const seen = new Set();
        for (const config of configGroups) {
            for (const run of runsWithExpectations[config]) {
                for (const exp of (run.expectations || [])) {

```



```

        if (!seen.has(exp.text)) {
            seen.add(exp.text);
            allAssertions.push(exp.text);
        }
    }
}

html += '<table class="benchmark-table" style="margin-top: 0.5rem;">';
html += "<thead><tr><th>Assertion</th>";
for (const config of configGroups) {
    const label = config.replace(/_/g, " ").replace(/\b\w/g, c =>
c.toUpperCase());
    html += "<th>" + escapeHtml(label) + "</th>";
}
html += "</tr></thead><tbody>";

for (const assertionText of allAssertions) {
    html += "<tr><td>" + escapeHtml(assertionText) + "</td>";

    for (const config of configGroups) {
        html += "<td>";
        for (const run of runsWithExpectations[config]) {
            const exp = (run.expectations || []).find(e => e.text ===
assertionText);
            if (exp) {
                const cls = exp.passed ? "benchmark-delta-positive" :
"benchmark-delta-negative";
                const icon = exp.passed ? "\u2713" : "\u2717";
                html += '<span class="' + cls + '" title="Run ' + run.run_number +
': ' + escapeHtml(exp.evidence || "") + '">' + icon + "</span> ";
            } else {
                html += "- ";
            }
        }
        html += "</td>";
    }
    html += "</tr>";
}
html += "</tbody></table>";
}
}

// Notes
if (notes.length > 0) {
    html += '<div class="benchmark-notes">';
    html += "<h3>Analysis Notes</h3>";
    html += "<ul>";
    for (const note of notes) {
        html += "<li>" + escapeHtml(note) + "</li>";
    }
    html += "</ul></div>";
}
}

```

```
        container.innerHTML = html;
    }

    // ---- Start ----
    init();
    renderBenchmark();
</script>
</body>
</html>
```